

Software Architecture and Design Specification

Project: Online Internship & Placement Preparation Portal

Version: 1.0

Authors: Vyshali Aytha, Vismaya Harish, Thejus A Reddy, Yashas Sadananda

Date: 14-09-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Vyshali Aytha, Vismaya Harish, Thejus A Reddy, Yashas Sadananda	Initial Software Architecture and Design Specification.

Approvals

Role	Name	Signature/Date
Coordinator	Pavithra P	

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Online Internship & Placement Preparation Portal, a college mini-project. It complements the Software Requirements Specification (SRS) by describing how the system is structured to satisfy those requirements: the chosen architectural pattern, component boundaries, data design, sequence flows, API design, and security architecture.

1.2 Scope

Covers the architecture and design for student registration and profile management, resume upload/download, internship/placement listing search and application, mock tests with auto-scoring, interview-question bank and mock-interview booking, recruiter job posting and shortlisting, and placement-cell approval/reporting. Institutional SSO, automated resume parsing, video-conferencing integration, payment-based premium test packs, and large-scale load handling are out of scope.

1.3 Audience

Student developers (the project team), evaluators/instructors, and anyone maintaining or extending the portal after submission.

1.4 Definitions

SRS, UI, API, OTP, JWT, CGPA, RTM, RBAC; RTM and RBAC are used additionally in this document.

2. Document Overview

2.1 How to use this document

Section 3 (Architecture) describes the system's structural decisions: goals, the chosen architectural pattern and its rationale, the component (UML) diagram, technology stack, risks, requirement traceability, and security architecture. Section 4 (Design) works at a finer grain: sequence diagrams for two representative flows, REST API definitions for three components, error handling/logging/monitoring conventions, and UX design notes.

2.2 Related Documents

SRS (v1.0), Online Internship & Placement Preparation Portal, including its Requirements Traceability Matrix (RTM).

3. Architecture

3.1 Goals & Constraints

Goals: usable role-based workflows for Student, Recruiter, and Placement Cell Admin; modular functional boundaries that map cleanly to the portal's eight functional modules; centrally enforced security (RBAC, session validation); acceptable responsiveness under the mini-project's expected load.

Constraints: single application server and a single relational database; no third-party integration beyond an Email/SMS gateway for OTP delivery; modern-browser web UI only, no native apps; HTTPS (TLS) required for all client-server communication.

3.2 Stakeholders & Concerns

- Students: a simple, reliable application flow and timely status updates.
- Recruiters: an easy way to post listings and shortlist applicants with minimal training.
- Placement Cell Admin: control over recruiter approval, oversight of listings, and placement reporting.
- Developers (project team): modularity and a small enough footprint to build and evaluate within one academic term.
- Evaluators/Instructors: a clear, traceable architecture that maps visibly back to the functional requirements and RTM.

3.3 Component (UML) Diagram

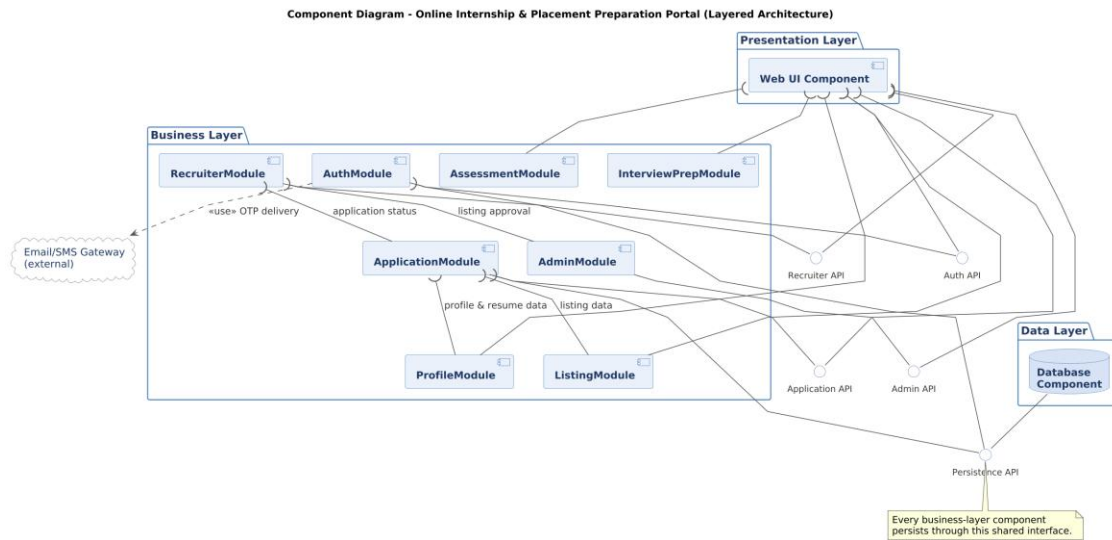


Figure 1 — Component diagram of the Online Internship & Placement Preparation Portal (Layered Architecture).

3.4 Component Descriptions

Component	Responsibility
Web UI Component	Responsive, role-based web dashboard for Student, Recruiter, and Admin views; routes all requests through the API layer.
AuthModule	Student self-registration with OTP, login/session issuance (JWT), account lockout after repeated failures, and recruiter account requests.
ProfileModule	Student profile fields, resume upload/replace, and generated PDF resume.
ListingModule	Displays active internship/placement listings and filters them by role, location, and CGPA eligibility.
ApplicationModule	Application submission with duplicate prevention, and the student's application status tracker.
AssessmentModule	Timed multiple-choice mock tests with automatic scoring and score history.
InterviewPrepModule	Searchable interview-question bank; mock-interview slot booking and Admin-recorded text feedback.
RecruiterModule	Listing creation (post admin approval) and applicant viewing/shortlisting.
AdminModule	Recruiter account approval and placement summary reporting.
Database (Data Layer)	Single relational database holding all persistent data — accounts, profiles, listings, applications, assessments, and bookings.

3.5 Chosen Architecture Pattern and Rationale

Layered Architecture was chosen —Presentation Layer containing the Web UI and API interface boundary, Business Layer containing the eight functional modules above, and a Data Layer (single relational database) — over Microservices and plain Client-Server, for two reasons:

- Small team, single-server, one-term scope: the system is built and evaluated within a single academic term by a small student team on a single application server. Layered Architecture's clear separation of UI, business rules, and persistence lets a small team divide and understand the codebase quickly, without the deployment and inter-service networking overhead Microservices would add at this scale.
- The requirements already form clean, co-located modules: the portal's own functional breakdown and RTM define eight cleanly bounded modules that share one database and one set of roles (Student, Recruiter, Admin). This maps directly onto a single Business Layer between one Presentation Layer and one Data Layer — exactly the shape Layered Architecture provides.

3.6 Technology Stack & Data Stores

Suggested implementation, consistent with the project's technology-agnostic requirements and single-server/single-database constraint: a responsive web front end (role-based views for Student/Recruiter/Admin) calling a REST/JSON API layer (e.g. Node.js/Express or Spring Boot); a single relational database (e.g. PostgreSQL or MySQL) for all persistent data; JWT-based session tokens; TLS/HTTPS for all client-server traffic; and a standard Email/SMS gateway API used only for OTP delivery.

3.7 Risks & Mitigations

Risk	Mitigation
Single application server is a single point of failure.	Accepted for the mini-project's scale; recommend regular database backups and basic uptime monitoring (OIPP-NF-002).
Fake or low-quality recruiter/listing postings.	Every listing requires prior Admin approval of the recruiter account before it can go live (OIPP-F-016 depends on OIPP-F-018).
Duplicate or accidental repeat applications.	ApplicationModule rejects a second application to the same listing (OIPP-F-010).
Brute-force login attempts.	Account lockout after 5 consecutive failed attempts for a defined lockout period (OIPP-F-003).
Unauthorized access to another user's data.	Role-based access control and JWT session validation enforced centrally at the API layer (OIPP-SR-002, OIPP-SR-005).

3.8 Traceability to Requirements

Requirement	Component
OIPP-F-001 Register with OTP	AuthModule
OIPP-F-003 Lockout after 5 failures	AuthModule
OIPP-F-005 Resume upload/replace	ProfileModule
OIPP-F-007/008 List & filter listings	ListingModule
OIPP-F-010 Apply, prevent duplicates	ApplicationModule
OIPP-F-012 Mock test auto-scoring	AssessmentModule
OIPP-F-015 Mock-interview booking	InterviewPrepModule
OIPP-F-016/017 Post listing, shortlist	RecruiterModule
OIPP-F-018/019 Approve recruiter, report	AdminModule

Full requirement-level traceability is maintained in the Requirements Traceability Matrix; this table summarizes the module each requirement group belongs to.

3.9 Security Architecture

Threat modeling (STRIDE), mapped to the project's security requirements (OIPP-SR-001–005):

Threat	Mitigation
Spoofing	Password hashing and JWT-based session validation on every authenticated request (OIPP-SR-003, OIPP-SR-005).
Tampering	HTTPS (TLS) enforced for all client–server communication (OIPP-SR-001).
Repudiation	Security-relevant events (logins, approvals, lockouts) are logged (OIPP-SR-004).
Information Disclosure	HTTPS in transit, plus role-based access control so a user can only read data their role permits (OIPP-SR-001, OIPP-SR-002).
Denial of Service	Not a primary requirement at mini-project scale; account lockout (OIPP-F-003) limits brute-force login attempts as a partial mitigation.
Elevation of Privilege	Role-based access control (RBAC) restricts each endpoint to its permitted role(s) — Student, Recruiter, or Admin (OIPP-SR-002).

4. Design

4.1 Design Overview

The portal is designed with layered modules for separation of concerns and easier maintenance, directly mirroring the portal's own functional breakdown: each functional area corresponds to exactly one Business-Layer component (see 3.4/3.8 above), so a change to one requirement area stays contained to its owning module.

4.2 UML Sequence Diagrams

Two flows specific to this project, covering the Student path and the Recruiter/Admin path respectively:

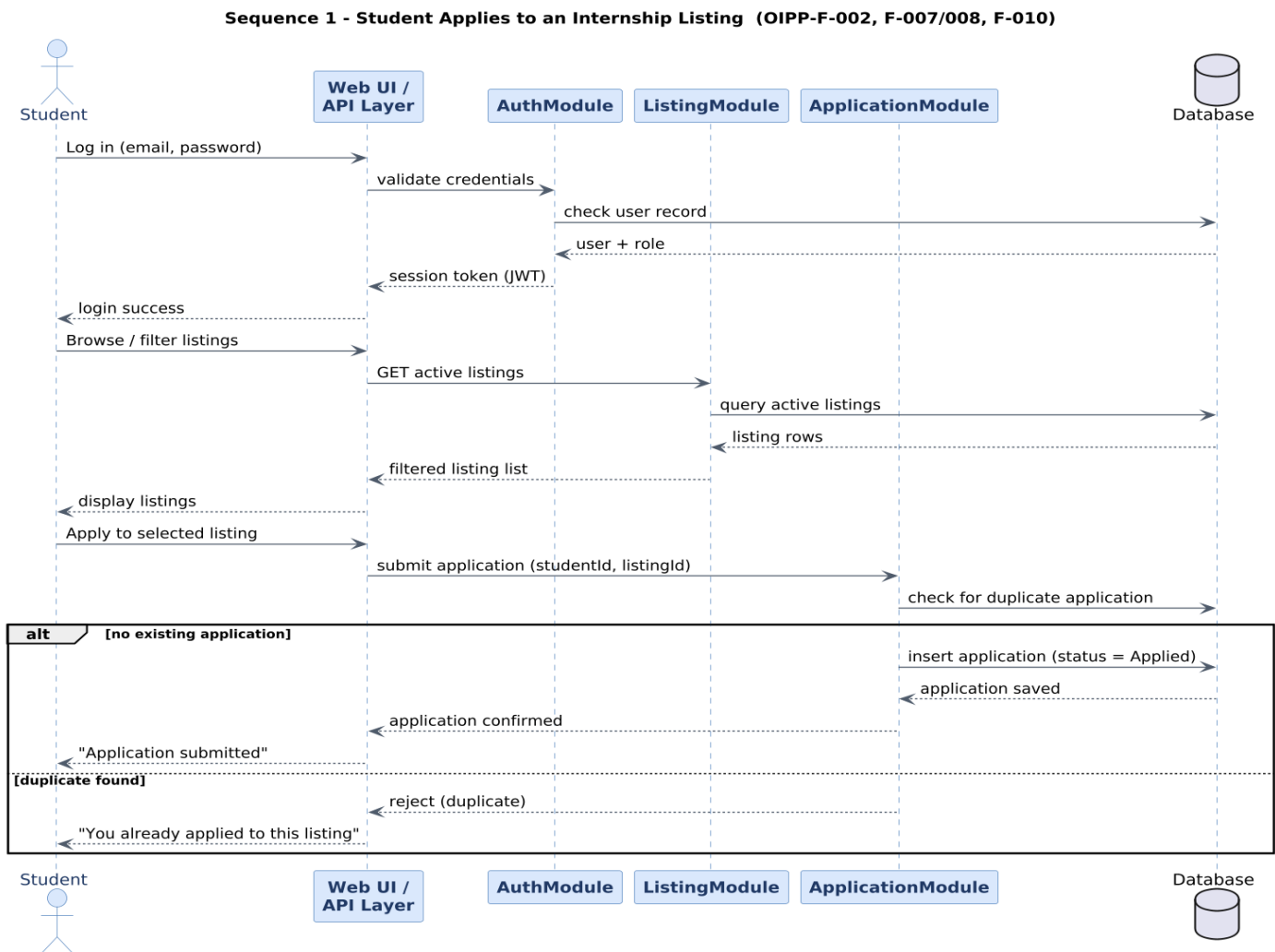


Figure 2 — Sequence: Student logs in, browses listings, and applies (OIPP-F-002, F-007/008, F-010).

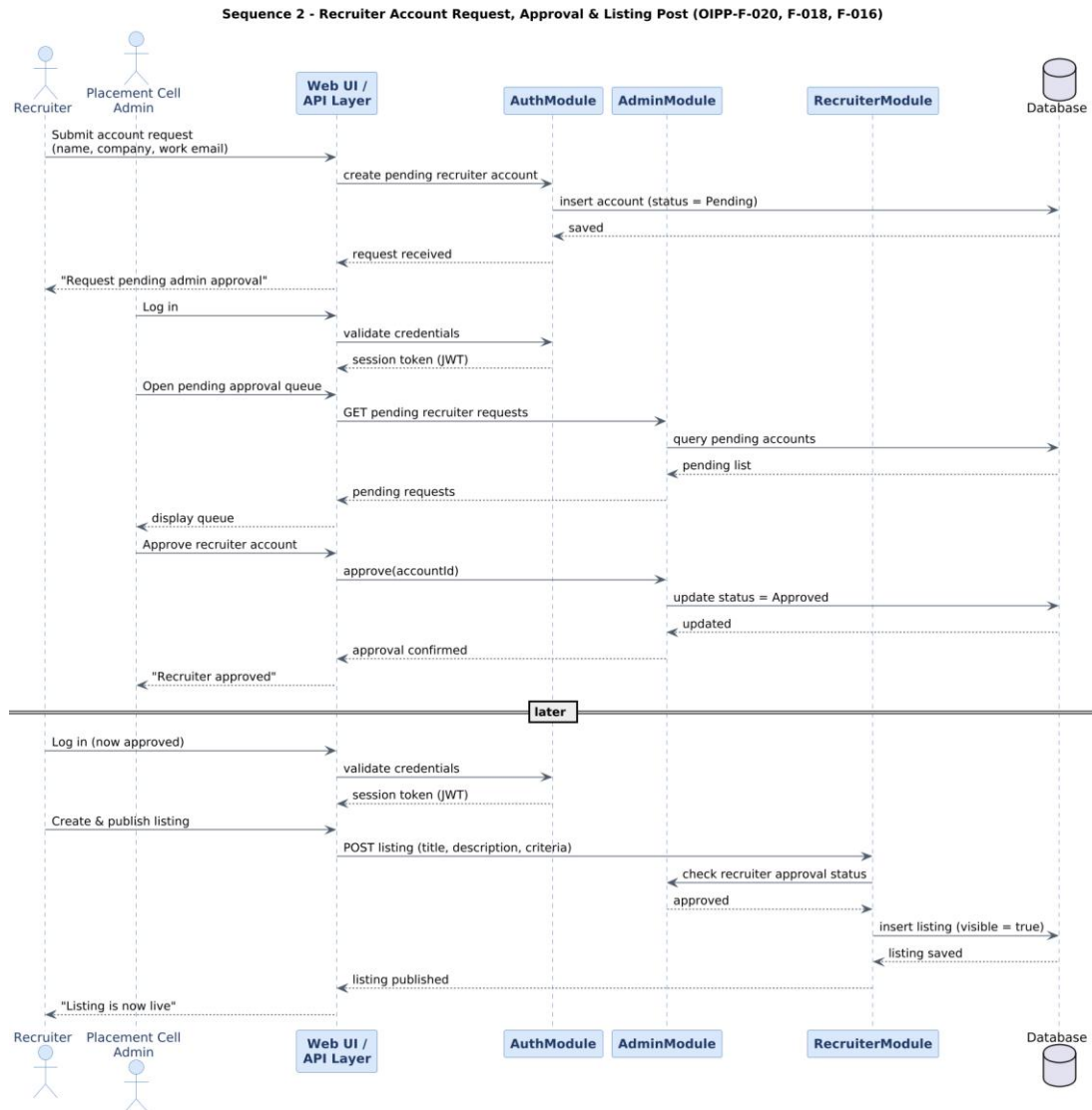


Figure 3 — Sequence: Recruiter account request & Admin approval, then listing post (OIPP-F-020, F-018, F-016).

4.3 API Design

Interface definitions for three components (AuthModule, ApplicationModule, InterviewPrepModule):

Component: AuthModule

Endpoint: POST /api/auth/register

Method: POST

Request: {name, email, password}

Response: {status, message: "OTP sent"}

Errors: 400 Invalid email, 409 Email already registered

Endpoint: POST /api/auth/verify-otp

Method: POST

Request: {email, otp}
Response: {status, message}
Errors: 400 Invalid or expired OTP

Component: ApplicationModule

Endpoint: POST /api/applications
Method: POST
Request: {studentId, listingId}
Response: {applicationId, status}
Errors: 401 Not authenticated, 404 Listing not found, 409 Duplicate application

Component: InterviewPrepModule

Endpoint: POST /api/interview-slots/{slotId}/book
Method: POST
Request: {studentId}
Response: {bookingId, status}
Errors: 401 Not authenticated, 404 Slot not found, 409 Slot already booked

4.4 Error Handling, Logging & Monitoring

Standardized error responses (HTTP status + short message) across all endpoints, matching the project's Pass/Fail acceptance-criteria pattern. No passwords, tokens, or other sensitive data appear in logs; security-relevant events (login, logout, approval) are logged per OIPP-SR-004. Monitoring covers response time against the OIPP-NF-001 target, uptime/availability (OIPP-NF-002), and concurrent-user load against the OIPP-NF-004 target.

4.5 UX Design

A single responsive web dashboard with role-based views for Student, Recruiter, and Admin, usable on both desktop and mobile browsers. Accessibility follows OIPP-NF-005: clear labels and adequate colour contrast across all views.

4.6 Open Issues & Next Steps

Future enhancements beyond this mini-project's scope: institutional single-sign-on (SSO), automated resume parsing, integrated video-conferencing for mock interviews, payment-based premium test packs, and large-scale (thousands of concurrent users) load handling.

5. Appendices

5.1 Glossary: SRS, UI, API, OTP, JWT, CGPA, RTM, RBAC.

5.2 References: SRS (Online Internship & Placement Preparation Portal); IEEE 42010; OWASP Top 10; WCAG 2.1 (accessibility, OIPP-NF-005).

5.3 Tools: PlantUML, draw.io.